

Фабрика гипотез

Объяснимый, воспроизводимый портфель
проверяемых R&D-гипотез с трассировкой до источников

Кейс. Флотационное обогащение Cu–Ni руд: снижение потерь ценных металлов с отвальными хвостами обогатительных фабрик. На вход — реальный отчёт института по хвостам (xlsx), корпус литературы (учебники кейса + открытые источники) и целевой KPI. На выходе — диагноз потерь в тоннах и деньгах плюс ранжированный портфель проверяемых гипотез с обоснованием, экономикой, планом эксперимента и трассировкой каждого числа до первоисточника.

Тип системы	Детерминированная платформа принятия R&D-решений, не чат с LLM
Архитектура	Монорепо: Rust-ядро · Python-сайдкар · React-фронт · Domain Pack
Данные кейса	4 фабрики (КГМК, НОФ-ВКР, НОФ-мед, ТОФ), 27 экспертных гипотез
Запуск	<code>docker compose up --build</code> — без <code>.env</code> , LLM-ключей и Postgres
Целевая поставка	Десктоп-приложение (Tauri): нативный выбор папки, скан корпуса

frontend :80 · backend API :8080 · sidecar API :8766 · опционально Postgres/pgvector

Содержание

1	Задача, кейс и данные	3
1.1	Исходные данные кейса	3
2	Ключевая идея	3
3	Архитектура: граница волатильности	4
3.1	Контракт границы	4
3.2	Три хранилища знаний	4
4	Сквозной пайплайн	4
5	Диагностика потерь — детерминированно, без LLM	5
5.1	Что вычисляется для каждой ячейки	5
5.2	Data Readiness — честность о качестве данных	5
6	Извлечение знаний — RAG под валидатором	5
7	Граф знаний и snapshot	6
8	Discovery Engine — детерминированный поиск гипотез	6
9	Скоринг и экономика	6
9.1	Экономический эффект	6
9.2	Шесть осей скоринга	7
9.3	Хард-фильтры и skeptic-правила	7
9.4	План эксперимента (DOE)	7
10	Roadmap и честная де-дубликация денег	7
11	Benchmark против экспертов — встроенная валидация	8
11.1	Мультифабричная карта и универсальность	8
12	Управление портфелем (regun)	8
13	Domain Pack — домен как данные	8
14	Контракты данных	9
15	API платформы	10
16	Фронтенд и экраны	10
17	Стек, запуск и режимы	11
17.1	Запуск	11
17.2	Режимы LLM	11
17.3	Технологии по слоям	11
18	CI, безопасность, качество	11

1. Задача, кейс и данные

Обогащительные фабрики теряют ценные металлы (в данных кейса анонимизированы как `element_28` / `element_29`) с отвальными хвостами. Причины потерь разнородны и рассредоточены: часть металла уходит в закрытых сростках (проблема измельчения), часть — в шлаках из-за переизмельчения, часть не извлекается текущим режимом флотации, часть неизвлекаема в принципе. Знания о том, как снижать потери, разбросаны по отчётам института, учебникам и открытым статьям.

Задача — превратить этот разрозненный материал в приоритизированный, экономически обоснованный и проверяемый план исследований, где каждое утверждение прослеживается до источника, а результат воспроизводим.

1.1 Исходные данные кейса

- **Отчёты по хвостам** — реальные `xlsx`-файлы 4 фабрик (`norn-hack/Пример 1..4/`): матрица потерь [класс крупности × минералогическая форма] с тоннажом. Структура файлов *плавает*: КГМК/НОФ — одна секция породных хвостов, ТОФ — породные и пирротинные.
- **Корпус литературы** — учебники по флотации (PDF) плюс открытые статьи (open access, с URL и лицензией) для оценки novelty.
- **Эталон экспертов** — 27 гипотез реальных мозговых штурмов по 4 фабрикам (`golden/expert_hypotheses.j`) как ground truth для встроенного benchmark.

Цены металлов на демо вводит пользователь (анонимизация данных сохраняется): диагноз в тоннах — из отчёта, перевод в деньги — по пользовательской цене.

2. Ключевая идея

Главное отличие от «чата с ИИ». Диагноз ставит детерминированный парсер. Гипотезы находит детерминированный алгоритм обхода графа знаний. Ранжирование, хард-фильтры и деньги считает детерминированный Rust-код. **LLM задействован только для чтения литературы** (извлечение фактов под строгим валидатором) и, опционально, для текстового объяснения результатов. LLM не принимает ни одного решения.

Из этого следуют четыре свойства, которых принципиально нет у чат-подхода:

Воспроизводимость	Те же данные + KPI + <code>rask</code> дают байт-в-байт тот же результат. Граф фактов строится <code>snapshot</code> -хешем; повторный прогон и <code>regun</code> по цене хеш не меняют.
Объяснимость	Каждое число прослеживается до ячейки исходного <code>xlsx</code> (<code>cell_ref</code>) или страницы источника (<code>source_page</code>) — кликабельный <code>trace</code> .
Экономика	Каждая гипотеза оценена в \$/год диапазоном с раскрытыми допущениями, а не абстрактным баллом.
Универсальность	Весь домен живёт в <code>YAML</code> -паке как данные; смена передела не трогает движок — переписывается только <code>сайдкар</code> и <code>rask</code> .

Почему не предсказательная ML-модель. 4 фабрики — это не датасет для обучения. Граф знаний с детерминированными операторами даёт объяснимость и воспроизводимость, которых требует постановка задачи; ML-скоринг — в `roadmap`, когда накопится история фидбэка по гипотезам.

3. Архитектура: граница волатильности

Главный архитектурный закон (`docs/AGENTS.md`) — разделение стабильного детерминированного ядра и волатильной периферии. Смена задачи (другой передел, другой KPI) требует переписать только Python-сайдкар и `pack`; ядро остаётся нетронутым. Нарушение этой границы — формальное основание отклонить PR.

Слой	Роль	Ответственность
Rust <code>backend/</code>	Стабильное ядро	Discovery Engine и платформа. Работает над <i>generic-графом</i> : только типы рёбер и теги узлов; ни одного доменного слова в коде.
Python <code>agent-system/</code>	Волатильный сайдкар	LLM-извлечение, парсинг <code>xlsx/pdf</code> , эмбединги, RAG, <code>novelty</code> , <code>skeptic</code> , <code>narrate</code> . Отдаёт только JSON, ничего не решает.
TS / React <code>frontend/</code>	Витрина	Heatmap диагноза, портфель, карточка гипотезы, граф, <code>benchmark</code> . Работает и от фикстур без бэка.
Domain Pack <code>docs/packs/</code>	Данные	Вся доменная семантика: <code>node/edge</code> -типы, операторы, веса скоринга, правила диагнозов, синонимы, <code>hard/skeptic</code> -правила.

3.1 Контракт границы

Граница Python ↔ Rust — **только JSON**, Rust валидирует на входе (**без PyO3**). Единый источник структур — крейт `contracts`; TS-типы фронта согласованы с ним. Бэкенд — три крейта чистой архитектуры: `contracts` (структуры), `engine` (Discovery Engine, `petgraph`, без I/O и доменных слов), `platform` (ахум-API, порт 8080).

3.2 Три хранилища знаний

- Структурная БД — численные факты и эксперименты (значения, единицы, условия).
- Граф механизмов — фактор → механизм → свойство → KPI с источниками (основа `trace`).
- Вектор-индекс (`pgvector`) — чанки документов для поиска доказательств и `novelty`.

4. Сквозной пайплайн

```
norn-hack/Пример N/Хвосты*.xlsx
→ POST /diagnose (детерминированно, БЕЗ LLM)
→ DiagnosticsReport { loss_cells, diagnosis_summary, data_quality }

корпус *.pdf + sample_docs/open/*.pdf
→ POST /extract (LLM под pydantic-валидатором)
→ ExtractResponse { claims, entities, edges }

оба → Rust platform валидирует JSON
→ Graph Agent строит petgraph, диагноз-узлы получают addressable_tons
→ snapshot + hash (граница воспроизводимости)
→ engine::discover(graph, kpi_contract, pack)
→ BoardResponse { snapshot, diagnostics, hypotheses[] с economic_effect }
→ Frontend: heatmap + портфель + карточка + benchmark
```

Публичная точка входа движка ровно одна:

```
pub fn discover(graph: &Graph, contract: &KpiContract, pack: &DomainPack) -> Portfolio
```

5. Диагностика потерь — детерминированно, без LLM

Диагноз ставит **якорный парсер xlsx** (`app/pipeline/diagnose/parser.py`). Поскольку структура файлов плавает между фабриками, парсер ищет *маркерные строки* (секции породных / пирротиновых хвостов, строки классов крупности), а не фиксированные координаты. Он устойчив к битым ячейкам (`#REF!`) и разному числу секций; проверен на всех 4 фабриках.

5.1 Что вычисляется для каждой ячейки

1. **Канонизация формы минерала** по подстроке в ячейке: «раскрыт» → `open_pnt_cp`, «закрыт» → `closed_pnt_cp`, «силикат» → `silicate`, «примесь в пирротине», «миллерит» и т.д.
2. **Извлекаемость** по элементу из `pack`: `element_28` извлекаем в формах `{open/closed_pnt_cp, millerite}`, `element_29` — в `{open/closed_pnt_cp}`.
3. **Диагноз причины** по правилам из `pack` (порядок важен — первое сработавшее):

Диагноз	Условие	Смысл
<code>not_recoverable</code>	<code>recoverable = false</code>	Неизвлекаемо текущей технологией (силикаты, примесь в пирротине)
<code>liberation_deficit</code>	закрытые сrostки, класс <code>coarse/medium</code>	Недораскрытие — проблема измельчения/классификации
<code>slimes_overgrinding</code>	раскрыт, класс <code>fine</code> (-10 мкм)	Переизмельчение: минерал ушёл в шламы, плохо флотируется
<code>flotation_kinetics</code>	раскрыт, класс <code>coarse/medium</code>	Флотируемого размера, но не извлечён — режим/кинетика флотации

Группы крупности заданы в пакке: **coarse** `{+125, -125+71, +71, -71+45}`, **medium** `{-45+20, -20+10}`, **fine** `{-10}`. Каждая цифра несёт `cell_ref` — ссылку на ячейку исходного отчёта (например `Итог!E44`), поэтому любое число кликабельно до первоисточника.

5.2 Data Readiness — честность о качестве данных

Проблемы данных не скрываются, а агрегируются в отчёт `DataReadiness` (`readiness_pct, issues_by_type`). Пример КГМК: в файле секция `+125` содержит битые ячейки, парсер помечает `ref_error: 33` и продолжает работу детерминированно; итоговая готовность данных показывается в UI (в демо КГМК $\approx 62.5\%$).

6. Извлечение знаний — RAG под валидатором

`POST /extract` превращает корпус в структурированные факты:

- PDF/txt режется на чанки с номерами страниц (`Document Agent`), кодируется эмбедингами в вектор-индекс (`pgvector`, опционально).
- LLM извлекает **claims** вида **рычаг** → **механизм** → **эффект** с полярностью (`polarity`) и постраничной привязкой (`source_page`), строго по Pydantic-схеме — валидатор отсекает всё, что не соответствует контракту.
- **Entity Resolver** нормализует терминологию через словарь `synonyms` из `pack` + fuzzy-matching: гидроциклон / циклон / ГЦ-660 → одна сущность `hydrocyclone`; аналогично мельницы (МШР, МШЦ), реагенты, минералы (`Pnt`, `Cr`, пирротин).

- Для стабильного демо есть **mock-режим** — отдаёт зафиксированный `fixtures/extract_response.json`, что делает всю систему работоспособной офлайн без ключей и сети.

7. Граф знаний и snapshot

Rust-ядро (Graph Agent) строит из `ExtractResponse` и `DiagnosticsReport` направленный граф на `petgraph`: фактор → механизм → диагноз → KPI. При построении:

- диагноз-узлы (тег `diagnosis`) получают `addressable_tons` из `diagnosis_summary` диагностики;
- факторы (тег `controllable`) получают признак доступности и `capex_class` из конфига фабрики;
- рёбра несут ссылки на исходные `claims (source_claims)` — база `trace` и оценки «grounded».

Готовый граф фиксируется как **snapshot с хешем** — это граница воспроизводимости: одинаковый граф → одинаковый хеш → одинаковый портфель.

8. Discovery Engine — детерминированный поиск гипотез

Движок обходит граф *назад* от узлов KPI (совпадающих с метрикой цели контракта) набором generic-операторов. Операторы работают только над `EdgeType` и тегами узлов — они **не знают про флотацию**; доменная начинка приходит из `labels` и `pack`. Причинные рёбра механизм-пути: `Mechanism`, `Proxy`, `Substitution`.

Оператор	Что ищет	Пример гипотезы	Этап
<code>mechanism_path</code>	путь от KPI через диагноз с тоннажом до доступного рычага	«замени насадки ГЦ 12 → 8»	P0
<code>substitution</code>	альтернативный рычаг с тем же механизмом	«грохот вместо гидроциклона»	P0
<code>gap</code>	диагноз с существенным тоннажом, к которому не ведёт ни один доступный рычаг	«нужна магнитная сепарация» (новое оборудование, capex 3)	P0
<code>contradiction</code>	два <code>claim</code> с противоположной полярностью на одном ребре	«найди граничное условие»	P1
<code>analogy_transfer</code>	механизм покрыт у X, у Y с теми же тегами не покрыт	«перенеси приём из Си-цикла»	P1

Каждый кандидат (`Candidate`) несёт путь узлов, рёбра, управляемый фактор, диагноз-узел с тоннажом, `trace` и признаки `is_gap` / `n_partial_matches`. Исключённые пользователем факторы (`excluded_factors`) отсекаются на этом этапе.

9. Скоринг и экономика

9.1 Экономический эффект

Для гипотезы с диагноз-узлом:

$$\text{value}_{[lo,hi]} = \sum_{el} \text{addressable_tons}[el] \times \frac{\text{gain}_{[lo,hi]} \%}{100} \times \text{price}[el]$$

где `addressable_tons` — из диагностики, `price` — из `KpiContract` (вводит пользователь), а диапазон прироста извлечения `gain` по умолчанию `default_gain_pct_range = [5, 15]%` из `pack` (если у пути нет своего диапазона из `claims`). Каждое слагаемое сопровождается строкой `assumptions` — допущение раскрыто. Кандидат без диагнозов-узла получает `value = 0`.

9.2 Шесть осей скоринга

Пусть v_{mid} — середина `value_usd_range` гипотезы, v_{max} — максимум по портфелю, $\bar{c}$ — средняя `confidence claims`, n — число узлов пути. Оси (все в $[0, 1]$):

Ось	Формула
<code>kpi_impact</code>	$\frac{v_{mid}}{v_{max}} \cdot \bar{c}_{path} \cdot (1 - 0.05 \max(0, n - 2))$ — эффект $\propto$ деньгам, нормирован на портфель
<code>evidence</code>	$\bar{c}_{trace} \cdot \min(1, trace /3)$
<code>plausibility</code>	доля «grounded» рёбер: $\#\{\text{рёбер с grounded-claim}\} / \#\text{рёбер}$
<code>cost</code>	по сарех-классу рычага: $1 \rightarrow 1.0, 2 \rightarrow 0.7, 3 \rightarrow 0.35$ (неизв. $\rightarrow 0.7$)
<code>risk</code>	$\text{clamp}(1 - 0.1 \text{missing_evidence} - 0.15 n_{contra}, 0, 1)$
<code>novelty</code>	gap: $\text{clamp}(0.8 - 0.1 n_{partial})$; иначе 0.3

Итоговый балл — взвешенная сумма с весами из `pack`: $\text{score_total} = \sum_d w_d \cdot \text{score}_d$, где по умолчанию `kpi_impact` 0.35, `evidence` 0.20, `plausibility` 0.15, `cost` 0.15, `risk` 0.10, `novelty` 0.05.

9.3 Хард-фильтры и skeptic-правила

Хард-ограничения из `pack` и контракта (например `sarrex_class` ≤ 3) отсекают недопустимые гипотезы до ранжирования. Skeptic-правила помечают слабые места:

- `uncovered_constraint` — есть `constraint` на свойство, но ни одного `claim` по нему;
- `low_evidence_top_candidate` — гипотеза в топ-3 с `evidence` < 0.55 ;
- `sarrex_domination` — существует гипотеза с сопоставимым эффектом при меньших затратах.

9.4 План эксперимента (DOE)

Для каждой гипотезы генерируется базовый `DoePlan`: цель, факторы (по `controllable`-узлам пути), измерения (KPI, kpi-проху, дельта стоимости) и минимальное число прогонов $= 3 \times \#\text{факторов}$.

10. Roadmap и честная де-дубликация денег

Наивно суммировать \$-эффекты всех гипотез нельзя: многие делят один и тот же тоннаж диагноза — это двойной счёт. GET /roadmap группирует гипотезы по *механизму потери* и считает честный совокупный эффект.

Пример КГМК: наивная сумма по 11 гипотезам $\approx$ \$87М (двойной счёт), честный `total_value_usd_range` $\approx$ [\$4.25М, \$12.76М] по 2 разным механизмам потерь — и все действия относятся к **sarrex 1** (быстрые настройки режима, без капзатрат). Параметр `?max_sarrex=1` работает как «бюджетный ползунок».

11. Benchmark против экспертов — встроенная валидация

В данных кейса есть 27 гипотез реальных экспертных мозговых штурмов по 4 фабрикам. Система использует их как ground truth: GET /benchmark показывает, сколько экспертных гипотез *воспроизведено* и какие *новые* найдены (подтверждённые литературой).

На КГМК движок переоткрывает **все 5** эталонных гипотез экспертов: `coverage_pct: 100`, `matched: 5`, `expert_total: 5` — за минуты против недели экспертной работы. Плюс гар-гипотезы (магнитная сепарация, тонкое грохочение), которые эксперты предлагали *добавить* как новое оборудование, система честно помечает как гар (оборудования нет на фабрике).

11.1 Мультифабричная карта и универсальность

GET /factories даёт карту возможностей по всем 4 фабрикам, например: KGMK $\approx$ \$16.5M / покрытие 100%; TOF $\approx$ \$17M (2 секции хвостов); NOF-мед $\approx$ \$9.2M / 87.5%; NOF-ВКР $\approx$ \$5.4M / 83.3%. Другая фабрика — другая структура отчёта — *тот же движок*: переключение делается сменой одного YAML.

12. Управление портфелем (rerun)

Портфель — интерактивный. Структурированные действия пересчитывают ранжирование за секунды с подсветкой изменений, **не меняя snapshot-хеш** (граф тот же, меняются только деньги/веса/фильтры):

Действие	Эффект
change_price	изменить цену металла → пересчёт value_usd_range и ранжирования
exclude_factor	исключить рычаг из рассмотрения
change_weight	изменить вес оси скоринга
add_constraint / relax_constraint	добавить / ослабить ограничение (например capex)

Пример: change_price для element_28 с 16500 до 33000 \$/т удваивает value_usd_range топ-гипотезы, а snapshot.hash остаётся прежним.

13. Domain Pack — домен как данные

Ключ к универсальности: вся доменная семантика вынесена в один YAML-файл (packs/flotation-v1.yaml), который читают и сайдкар, и движок. В коде ядра нет слов «флотация», «гидроциклон», «пентландит». Что живёт в паке:

node_types	типы узлов (factor/mechanism/property/kpi) и теги (controllable, kpi_proxy, diagnosis, constraint)
extra_tags	
edge_types	mechanism, tradeoff, substitution, proxy
operators	какие операторы включены (mechanism_path, substitution, gap)
default_gain_pct_range	консервативный диапазон прироста извлечения [5, 15]%
scoring_weights	веса 6 осей скоринга
hard_constraints	жёсткие фильтры (capex_class ≤ 3)
skeptical_rules	правила критики гипотез
diagnosis_config	группы крупности, извлекаемость по элементам, правила диагнозов
synonyms	словарь нормализации терминологии для Entity Resolver

Второй pack (**metallurgy-v1.yaml**) демонстрирует переносимость подхода на смежный передел. Конфиги фабрик (**factories/*.yaml**) описывают доступное оборудование: у КГМК магнитная сепарация и тонкое грохочение помечены **present: false** — движок использует это, чтобы отличать реализуемые рычаги от гар-гипотез «нужно новое оборудование».

14. Контракты данных

Единый источник структур — крейт **contracts**. Ключевые типы:

Структура	Назначение
ExtractResponse	claims + entities + edges от сайдкара
Claim / EvidenceType	факт с source_page, confidence, polarity, типом доказательства
DiagnosticsReport	loss_cells, totals, diagnosis_summary, data_quality
LossCell / DataQualityIssue	ячейка потерь с cell_ref; проблема данных
GraphNode / GraphEdge	узлы и рёбра generic-графа
KpiContract / Target / Constraint	цель, ограничения, цены металлов
EconomicEffect	addressable_tons, gain-range, value_usd_range, assumptions
ScoreBreakdown	6 осей скоринга
Hypothesis / Status	гипотеза с trace, score, DOE, статусом
BenchmarkReport / ExpertMatch	сверка с экспертными гипотезами
TraceReport / SourceCell	трассировка до claims и ячеек xlsx
RoadmapPlan / RoadmapPhase	план с честной де-дубликацией
Snapshot / BoardResponse	хеш графа и полная доска портфеля
RerunAction / RerunKind	интерактивные действия над портфелем

15. API платформы

Эндпоинт	Назначение
POST /run	прогон discovery по фабрике/KPI → run_id + board
GET /board?run_id=	текущая доска портфеля
POST /rerun	изменить веса/цены/фильтры и пересчитать ранжирование
GET /hypothesis/:id	одна гипотеза целиком
GET /benchmark?run_id=	покрытие экспертных гипотез (обычно 100% на KGMK)
GET /data_readiness?run_id=	качество исходного xlsx (ref_error и т.п.)
GET /trace/:hyp_id?run_id=	трассировка до claims (страниц PDF) и ячеек xlsx
GET /roadmap?run_id=&max_capex=	план действий с честной де-дубликацией
GET /factories	мультифабричная карта денег по всем 4 фабрикам
GET /extract, /expert_hypotheses	сырое извлечение и эталон экспертов
GET /export/board.{json,csv}	выгрузка портфеля

Фронт ходит через vite/nginx-проxy /api на один origin (CORS в platform нет). Сайдкар (:8766) дополнительно даёт /diagnose, /extract, /retrieve, /embed, /novelty, /skeptical, /narrate, /health.

16. Фронтенд и экраны

React 19 + TypeScript, Vite 6, react-router 7, TanStack Query 5, CSS Modules. Весь доступ к данным — через единый ApiClient: при ошибке http-пути клиент падает обратно на фикстуры (демо-

страховка). Целевая поставка — десктоп-приложение на Tauri (нативный file-dialog, скан папки, спавн сайдкара); переход аддитивный, нативные вызовы за feature-detection `window.__TAURI__`.

Восемь экранов (переключатель фабрики в топбаре):

1. **Setup** — выбор фабрики, целевого элемента, цены, лимита CAPEX-класса.
2. **Run** — прогон пайплайна (сборка графа) с анимацией стадий.
3. **Diagnosis** — тепловая карта потерь [класс × минералогия], разбивка по диагнозам, data readiness.
4. **Hypotheses** — ранжированный портфель с деньгами, геун-контролы, детальный разбор гипотезы.
5. **Graph** — полный граф знаний: рычаги, механизмы, свойства, KPI.
6. **Benchmark** — сверка гипотез системы с экспертным штурмом.
7. **Library** — подключённая литература и данные, статус извлечения фактов.
8. **Report** — экспорт итогового портфеля (JSON / CSV).

17. Стек, запуск и режимы

17.1 Запуск

```
docker compose up --build → frontend http://localhost · backend :8080 · sidecar :8766
```

Стек по умолчанию поднимается **без .env, LLM-ключей и Postgres**: детерминированный /diagnose, mock-/extract и весь фронт работают из коробки. Контейнеры — read-only, no-new-privileges.

17.2 Режимы LLM

LLM задействован строго на двух этапах — **extract** (P0) и **narrate** (P1, объяснение гипотез). По умолчанию — mock-извлечение, LLM выключен. Для live-режима: **agent-system/.env** из примера, OPENAI_API_KEY или Yandex-credentials, **SIDECAR_LLM_ENABLED=true**. RAG/Postgres/pgvector — опция через профиль rag в `compose.yaml`.

17.3 Технологии по слоям

Rust	axum, petgraph, чистая архитектура (contracts/engine/platform), sqlite claim-store
Python 3.12	FastAPI + uvicorn, pydantic 2, openai-совместимый клиент (Yandex), pymupdf, python-docx, openpyxl, pycopg + pgvector
Frontend	React 19, TS, Vite 6, react-router 7, TanStack Query 5, Vitest, pnpm
Инфра	Docker Compose (frontend/backend/sidecar/postgres), Makefile, GitHub Actions

18. CI, безопасность, качество

GitHub Actions по сервису со своим path-фильтром:

- **Frontend** — lint, typecheck, тесты с покрытием, build, docker + smoke.

- **Backend** — `cargo fmt --check`, `clippy -D warnings`, `cargo test`, `docker`; golden-тест портфеля (`golden_board_flotation_v1`).
- **Agent System** — `ruff`, `мypy`, `compileall`, `pytest` (контракт-тесты `extract/retrieval`, `path-security`), `docker`.
- **Security** — `pnpm audit`, `CodeQL (SAST)`, `gitleaks`; плюс запуск по расписанию.

Dependabot еженедельно обновляет `npm/cargo/pip`-зависимости и GitHub Actions. Контракт-тест `verify:parity` сверяет фикстуры фронта с живым backend.

Итог

Фабрика гипотез превращает локальный корпус отчётов и литературы в объяснимый, воспроизводимый и экономически оценённый портфель R&D-экспериментов. Детерминированное ядро гарантирует, что каждое решение прослеживается до исходной ячейки или страницы и повторяемо байт-в-байт; граница волатильности и Domain Pack делают систему переносимой на любой передел сменой одного YAML-файла — без единой правки движка. LLM здесь читает литературу, но решений не принимает: диагноз, поиск гипотез, ранжирование и деньги считает детерминированный код.